CASE STUDY 1: Examination Result and Grace-Mark Validation

1. Problem Statement

Develop a program to process marks in five subjects, apply a limited grace-mark policy to eligible subject shortfalls, and determine the student's final classification. The program must prevent marks from exceeding the maximum and must not classify a student who has failed any subject.

2. Objectives

• Accept and validate marks for five subjects.

• Apply a maximum total grace allowance of five marks.

• Pass a subject when its adjusted mark is at least 35.

• Calculate total, percentage, and classification.

• Generate a clear result report.

3. Assumptions

• Five subjects, each out of 100.

• Minimum pass mark is 35 in every subject.

• At most five grace marks are available in total.

• Grace is applied only to a subject short by five marks or fewer, and only enough marks to reach 35.

• Grace is assigned in subject-entry order.

• Classification: Distinction ≥75%, First Class ≥60%, Second Class ≥50%, otherwise Pass Class; all subjects must be passed.

4. Algorithm

• Start.

• Read student name and five subject marks.

• Validate every mark is an integer from 0 to 100.

• For each subject below 35, calculate its shortfall.

• If shortfall is at most five and grace remains, apply the smaller of shortfall and remaining grace.

• Calculate adjusted total and percentage.

• If any subject remains below 35, report FAIL / Not Classified.

• Otherwise assign classification using percentage thresholds.

• Display original marks, grace applied, adjusted marks, and result.

• Stop.

5. Flowchart

The following traditional text flowchart shows the program sequence.

```
START
  |
Read student details and five marks
  |
Validate marks (0 to 100)
  |
Check each subject against 35
  |
Eligible shortfall (<=5) and grace remains?
  | YES: apply required grace
```

```
  | NO: leave marks unchanged
  |
Calculate adjusted total and percentage
  |
All subjects passed?
  | YES -> Assign class by percentage
  | NO  -> FAIL / NOT CLASSIFIED
  |
Display result
  |
 STOP
```

6. Pseudocode

```
BEGIN
  INPUT student name
  FOR each of five subjects
      INPUT mark
      VALIDATE 0 <= mark <= 100
  END FOR
  grace_remaining = 5
  FOR each subject mark
      shortfall = 35 - mark
      IF 0 < shortfall <= 5 AND grace_remaining > 0
          grace = MIN(shortfall, grace_remaining)
          mark = mark + grace
          grace_remaining = grace_remaining - grace
      END IF
  END FOR
  IF any adjusted mark < 35
      result = FAIL / NOT CLASSIFIED
  ELSE
      percentage = adjusted total / 500 * 100
      ASSIGN classification using thresholds
  END IF
  DISPLAY detailed report
END
```

7. Python Source Code

The following program is designed to run in Python 3.

```
PASS_MARK = 35
MAX_GRACE = 5
SUBJECTS = ("Python", "Mathematics", "Physics", "Chemistry", "English")

def read_mark(subject):
    while True:
        try:
            mark = int(input(f"Enter {subject} marks (0-100): "))
            if 0 <= mark <= 100:
                return mark
            print("Marks must be between 0 and 100.")
        except ValueError:
            print("Enter a whole number.")

def classify(percentage):
    if percentage >= 75:
        return "Distinction"
    if percentage >= 60:
        return "First Class"
    if percentage >= 50:
        return "Second Class"
    return "Pass Class"

def main():
    name = input("Enter student name: ").strip() or "Student"
    original = [read_mark(subject) for subject in SUBJECTS]
    adjusted = original.copy()
    grace_used = [0] * len(SUBJECTS)
    remaining = MAX_GRACE

    for i, mark in enumerate(adjusted):
        shortfall = PASS_MARK - mark
        if 0 < shortfall <= 5 and remaining > 0:
            applied = min(shortfall, remaining)
            adjusted[i] += applied
            grace_used[i] = applied
            remaining -= applied

    total = sum(adjusted)
    percentage = total / (len(SUBJECTS) * 100) * 100
    passed = all(mark >= PASS_MARK for mark in adjusted)

    print("\n========== EXAMINATION RESULT ==========")
    print(f"Student: {name}")
    for subject, before, grace, after in zip(
```

```
        SUBJECTS, original, grace_used, adjusted
    ):
        print(f"{subject:<14} Original={before:>3} Grace={grace:>2} Final={after:>3}")
    print(f"Grace used: {MAX_GRACE - remaining}/{MAX_GRACE}")
    print(f"Total: {total}/500")
    print(f"Percentage: {percentage:.2f}%")
    if passed:
        print(f"Result: PASS - {classify(percentage)}")
    else:
        print("Result: FAIL - NOT CLASSIFIED")

if __name__ == "__main__":
    main()
```

8. Test Cases

Test case	Input condition	Expected result
1	80, 75, 33, 70, 65	2 grace marks applied; 325/500; 65%; First Class.
2	80, 30, 32, 70, 65	Only five grace marks available; at least one subject remains failed.
3	100, 100, 100, 100, 100	500/500; 100%; Distinction.
4	Negative mark or mark above 100	Input rejected and requested again.

9. Sample Output

Illustrative execution:

```
Enter student name: Anu
Enter Python marks (0-100): 80
Enter Mathematics marks (0-100): 75
Enter Physics marks (0-100): 33
Enter Chemistry marks (0-100): 70
Enter English marks (0-100): 65

========== EXAMINATION RESULT ==========
Student: Anu
Python         Original= 80 Grace= 0 Final= 80
Mathematics    Original= 75 Grace= 0 Final= 75
Physics        Original= 33 Grace= 2 Final= 35
Chemistry      Original= 70 Grace= 0 Final= 70
English        Original= 65 Grace= 0 Final= 65
Grace used: 2/5
Total: 325/500
Percentage: 65.00%
Result: PASS - First Class
```

10. Conclusion

Grace marks are applied only within the stated policy. The program validates marks, prevents excessive grace allocation, and reports a classification only when every subject is passed.

CASE STUDY 12: Attendance Recovery Computation

1. Problem Statement

Create a program that determines how many additional consecutive classes a student must attend to reach the required attendance percentage, considering possible future absences.

2. Objectives

• Calculate current attendance percentage.

• Evaluate multiple future-absence scenarios.

• Calculate the minimum additional classes required to reach 75%.

• Validate attendance inputs and display results.

3. Assumptions

• Attendance threshold is 75%.

• Total classes conducted must be greater than zero.

• Attended classes must be between zero and total classes.

• Future absences are additional classes missed before the recovery period.

• Every recovery class is attended.

• The required number of recovery classes is rounded up to the next whole class.

4. Algorithm

• Read total and attended classes.

• Validate the values.

• Read the number of future-absence scenarios.

• For each scenario, read future absences.

• Calculate projected total classes and attended classes before recovery.

• Calculate the minimum additional classes using the 75% inequality.

• Display current percentage and recovery requirement for each scenario.

• Stop.

5. Flowchart

The following traditional text flowchart shows the program sequence.

```
START
  |
Read total classes and attended classes
  |
Validate attendance data
  |
Read future absence scenarios
  |
For each scenario:
  Calculate projected total and attended
  |
Calculate minimum recovery classes
  |
Display scenario results
  |
 STOP
```

6. Pseudocode

```
BEGIN
  INPUT total_classes, attended_classes
```

```
  VALIDATE total_classes > 0
  VALIDATE 0 <= attended_classes <= total_classes
  INPUT scenario_count
  FOR each scenario
      INPUT future_absences
      projected_total = total_classes + future_absences
      projected_attended = attended_classes
      required = CEILING(
          MAX(0, 0.75 * projected_total - projected_attended)
          / 0.25
      )
      DISPLAY required
  END FOR
END
```

7. Python Source Code

The following program is designed to run in Python 3.

```
import math

THRESHOLD = 0.75

def read_int(message, minimum=0):
    while True:
        try:
            value = int(input(message))
            if value >= minimum:
                return value
            print(f"Enter an integer greater than or equal to {minimum}.")
        except ValueError:
            print("Enter a valid integer.")

def calculate_recovery(total, attended, future_absences):
    projected_total = total + future_absences
    required = math.ceil(
        max(0, THRESHOLD * projected_total - attended)
        / (1 - THRESHOLD)
    )
    return required

def main():
    print("ATTENDANCE RECOVERY COMPUTATION")
    total = read_int("Enter total classes conducted: ", 1)
    attended = read_int("Enter classes attended: ")
    while attended > total:
        print("Attended classes cannot exceed total classes.")
        attended = read_int("Enter classes attended: ")

    count = read_int("Enter number of future absence scenarios: ", 1)
    print("\nFuture Attendance Scenarios")
    for index in range(1, count + 1):
        absence = read_int(f"Enter future absences for scenario {index}: ")
        current_percentage = attended / total * 100
        required = calculate_recovery(total, attended, absence)
        projected_total = total + absence
        final_percentage = (
            (attended + required) / (projected_total + required) * 100
        )
        print(
            f"Scenario {index}: Current={current_percentage:.2f}%, "
            f"Future absences={absence}, "
            f"Additional classes required={required}, "
            f"Resulting attendance={final_percentage:.2f}%"
        )

if __name__ == "__main__":
    main()
```

8. Test Cases

Test case	Input condition	Expected result
1	100 total, 65 attended, 0 future absences	40 additional classes; reaches 75%.
2	100 total, 65 attended, 5 future absences	55 additional classes.
3	100 total, 65 attended, 10 future absences	70 additional classes.
4	100 total, 80 attended, 0 future absences	0 additional classes required.
5	Attended greater than total	Input rejected.

9. Sample Output

Illustrative execution:

```
ATTENDANCE RECOVERY COMPUTATION
Enter total classes conducted: 100
Enter classes attended: 65
Enter number of future absence scenarios: 4

Future Attendance Scenarios
Enter future absences for scenario 1: 0
Scenario 1: Current=65.00%, Future absences=0, Additional classes required=40, Resulting attendance=75.00%
Enter future absences for scenario 2: 5
Scenario 2: Current=65.00%, Future absences=5, Additional classes required=55, Resulting attendance=75.00%
Enter future absences for scenario 3: 10
Scenario 3: Current=65.00%, Future absences=10, Additional classes required=70, Resulting attendance=75.00%
Enter future absences for scenario 4: 20
Scenario 4: Current=65.00%, Future absences=20, Additional classes required=100, Resulting attendance=75.00%
```

10. Conclusion

The program calculates the minimum number of classes that must be attended to restore attendance to the required threshold under different future-absence scenarios.

CASE STUDY 13: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate an ATM that processes deposits and withdrawals, enforces withdrawal limits, prevents overdrafts, records transaction history, and displays a final account summary.

2. Objectives

• Process deposits and withdrawals.

• Validate transaction amounts and types.

• Enforce withdrawal limits of Rs.100 to Rs.10,000.

• Prevent withdrawals exceeding the available balance.

• Maintain transaction history and summary totals.

3. Assumptions

• Initial balance is a non-negative whole-rupee amount.

• Transaction amounts must be positive integers.

• Minimum withdrawal is Rs.100 and maximum is Rs.10,000.

• Deposits have no upper limit in this simulation.

• Rejected transactions do not change the balance.

4. Algorithm

• Read initial balance and transaction count.

• For each transaction, read type and amount.

• Validate transaction type and amount.

• For deposits, add amount to balance.

• For withdrawals, check limits and available balance.

• Record successful or rejected transaction.

• Display transaction history and final totals.

• Stop.

5. Flowchart

The following traditional text flowchart shows the program sequence.

```
START
  |
Read opening balance and transaction count
  |
Read transaction type and amount
  |
Valid deposit?
  | YES -> Add to balance
  |
Valid withdrawal limits and sufficient funds?
  | YES -> Subtract from balance
  | NO  -> Reject transaction
  |
Record transaction
  |
More transactions?
  | YES -> Repeat
  | NO
  |
Display history and summary
  |
 STOP
```

6. Pseudocode

```
BEGIN
  INPUT opening_balance
  INPUT transaction_count
  FOR each transaction
      INPUT transaction_type, amount
      IF amount <= 0
          REJECT
      ELSE IF type = DEPOSIT
          balance = balance + amount
          RECORD SUCCESS
      ELSE IF type = WITHDRAWAL
          IF amount < 100 OR amount > 10000
              REJECT
          ELSE IF amount > balance
              REJECT
          ELSE
              balance = balance - amount
              RECORD SUCCESS
          END IF
      ELSE
          REJECT INVALID TYPE
      END IF
  END FOR
  DISPLAY transaction history and summary
END
```

7. Python Source Code

The following program is designed to run in Python 3.

```
def read_non_negative(message):
    while True:
        try:
            value = int(input(message))
            if value >= 0:
                return value
            print("Value cannot be negative.")
        except ValueError:
            print("Enter a whole number.")

def process_transaction(balance, kind, amount):
    if amount <= 0:
        return balance, False, "Amount must be positive."
    if kind == "D":
        return balance + amount, True, "Deposit successful."
    if kind == "W":
        if amount < 100 or amount > 10000:
            return balance, False, "Withdrawal must be Rs.100 to Rs.10,000."
        if amount > balance:
            return balance, False, "Insufficient balance."
        return balance - amount, True, "Withdrawal successful."
    return balance, False, "Invalid transaction type."

def main():
    print("ATM TRANSACTION SEQUENCE SIMULATION")
    balance = read_non_negative("Enter initial balance: Rs.")
    count = read_non_negative("Enter number of transactions: ")
    history = []
    deposits = withdrawals = successful = rejected = 0

    for number in range(1, count + 1):
        print(f"\nTransaction {number}")
        kind = input("Enter D for deposit or W for withdrawal: ").strip().upper()
        amount = read_non_negative("Enter amount: Rs.")
        old_balance = balance
        balance, ok, message = process_transaction(balance, kind, amount)
        if ok:
            successful += 1
            if kind == "D":
                deposits += amount
            else:
                withdrawals += amount
        else:
            rejected += 1
        history.append((number, kind, amount, message, balance))
        print(message)

    print("\n========== ATM TRANSACTION SUMMARY ==========")
    for number, kind, amount, message, closing in history:
        print(f"{number}. {kind} Rs.{amount} - {message} Balance: Rs.{closing}")
    print(f"Total deposits: Rs.{deposits}")
    print(f"Total withdrawals: Rs.{withdrawals}")
    print(f"Successful transactions: {successful}")
    print(f"Rejected transactions: {rejected}")
    print(f"Final balance: Rs.{balance}")
```

```
if __name__ == "__main__":
    main()
```

8. Test Cases

Test case	Input condition	Expected result
1	Deposit Rs.2,000 into Rs.5,000 balance	Balance becomes Rs.7,000.
2	Withdraw Rs.1,500 from Rs.7,000	Balance becomes Rs.5,500.
3	Withdraw Rs.8,000 from Rs.5,500	Rejected: insufficient balance.
4	Withdraw Rs.50	Rejected: below minimum withdrawal.
5	Invalid transaction type	Rejected without changing balance.

9. Sample Output

Illustrative execution:

```
ATM TRANSACTION SEQUENCE SIMULATION
Enter initial balance: Rs.5000
Enter number of transactions: 5

Transaction 1
Enter D for deposit or W for withdrawal: D
Enter amount: Rs.2000
Deposit successful.

Transaction 2
Enter D for deposit or W for withdrawal: W
Enter amount: Rs.1500
Withdrawal successful.

Transaction 3
Enter D for deposit or W for withdrawal: W
Enter amount: Rs.8000
Insufficient balance.

Transaction 4
Enter D for deposit or W for withdrawal: W
Enter amount: Rs.50
Withdrawal must be Rs.100 to Rs.10,000.

Transaction 5
Enter D for deposit or W for withdrawal: X
Enter amount: Rs.500
Invalid transaction type.

========== ATM TRANSACTION SUMMARY ==========
1. D Rs.2000 - Deposit successful. Balance: Rs.7000
2. W Rs.1500 - Withdrawal successful. Balance: Rs.5500
3. W Rs.8000 - Insufficient balance. Balance: Rs.5500
4. W Rs.50 - Withdrawal must be Rs.100 to Rs.10,000. Balance: Rs.5500
5. X Rs.500 - Invalid transaction type. Balance: Rs.5500
Total deposits: Rs.2000
Total withdrawals: Rs.1500
Successful transactions: 2
Rejected transactions: 3
Final balance: Rs.5500
```

10. Conclusion

The ATM simulation processes transactions safely, enforces withdrawal rules, prevents overdrafts, and produces a transaction history with a final balance.

CASE STUDY 4: Warehouse Inventory Reconciliation

1. Problem Statement

Compare opening stock, incoming quantities, dispatched quantities, and physically counted closing stock for multiple products. Identify shortages and excesses and calculate replenishment requirements.

2. Objectives

• Record product-wise stock movement.

• Calculate expected closing stock.

• Compare expected and physical closing stock.

• Identify shortage, excess, or matched stock.

• Calculate replenishment quantity against minimum stock.

3. Assumptions

• All quantities are non-negative integers.

• Dispatched quantity cannot exceed opening plus incoming stock.

• Product names are unique.

• Expected closing = opening + incoming - dispatched.

• Discrepancy = actual closing - expected closing.

• Replenishment = max(0, minimum stock - actual closing).

4. Algorithm

• Read number of products.

• For each product, read opening, incoming, dispatched, actual closing, and minimum stock.

• Validate all quantities and dispatch limit.

• Calculate expected closing stock and discrepancy.

• Classify stock status.

• Calculate replenishment requirement.

• Display product-wise and total summary.

5. Flowchart

The following traditional text flowchart shows the program sequence.

```
START
  |
Read product count
  |
Read product stock details
  |
Validate quantities and dispatch limit
  |
Calculate expected closing stock
  |
Compare actual and expected stock
  |
Classify shortage / excess / matched
  |
Calculate replenishment
  |
More products?
  | YES -> Repeat
  | NO
  |
Display reconciliation report
  |
 STOP
```

6. Pseudocode

```
BEGIN
  INPUT product_count
  FOR each product
      INPUT name, opening, incoming, dispatched
      INPUT actual_closing, minimum_stock
      VALIDATE all quantities >= 0
      VALIDATE dispatched <= opening + incoming
      expected = opening + incoming - dispatched
      discrepancy = actual_closing - expected
      replenishment = MAX(0, minimum_stock - actual_closing)
      IF discrepancy < 0: status = SHORTAGE
      ELSE IF discrepancy > 0: status = EXCESS
      ELSE: status = MATCHED
      STORE product report
  END FOR
  DISPLAY report and totals
END
```

7. Python Source Code

The following program is designed to run in Python 3.

```
def read_qty(message):
    while True:
        try:
            value = int(input(message))
            if value >= 0:
                return value
            print("Quantity cannot be negative.")
        except ValueError:
            print("Enter a whole number.")

def main():
    print("WAREHOUSE INVENTORY RECONCILIATION")
    count = read_qty("Enter number of products: ")
    records = []
    names = set()

    for i in range(count):
        print(f"\nPRODUCT {i + 1}")
        while True:
            name = input("Product name: ").strip()
            if name and name.casefold() not in names:
                names.add(name.casefold())
                break
            print("Name must be non-empty and unique.")
        opening = read_qty("Opening stock: ")
        incoming = read_qty("Incoming quantity: ")
        while True:
            dispatched = read_qty("Dispatched quantity: ")
            if dispatched <= opening + incoming:
                break
            print("Dispatch exceeds available stock.")
        actual = read_qty("Actual closing stock: ")
        minimum = read_qty("Minimum stock level: ")

        expected = opening + incoming - dispatched
        discrepancy = actual - expected
        replenishment = max(0, minimum - actual)
        status = ("SHORTAGE" if discrepancy < 0 else
                  "EXCESS" if discrepancy > 0 else "MATCHED")
        records.append((name, expected, actual, discrepancy,
                        replenishment, status))

    print("\n========== INVENTORY RECONCILIATION ==========")
    for name, expected, actual, difference, replenish, status in records:
        print(f"{name}: Expected={expected}, Actual={actual}, "
              f"Difference={difference}, Replenish={replenish}, {status}")
    print(f"Total expected stock: {sum(r[1] for r in records)}")
    print(f"Total actual stock: {sum(r[2] for r in records)}")
    print(f"Total replenishment required: {sum(r[4] for r in records)}")

if __name__ == "__main__":
    main()
```

8. Test Cases

Test case	Input condition	Expected result
1	Opening 50, incoming 20, dispatched 30, actual 40, minimum 35	Expected 40; matched; replenishment 0.

Test case	Input condition	Expected result
2	Opening 50, incoming 20, dispatched 30, actual 38, minimum 45	Expected 40; shortage 2; replenishment 7.
3	Opening 100, incoming 50, dispatched 80, actual 75, minimum 60	Expected 70; excess 5; replenishment 0.
4	Dispatched quantity exceeds opening plus incoming	Input rejected.

9. Sample Output

Illustrative execution:

```
WAREHOUSE INVENTORY RECONCILIATION
Enter number of products: 3

PRODUCT 1
Product name: Laptop
Opening stock: 50
Incoming quantity: 20
Dispatched quantity: 30
Actual closing stock: 38
Minimum stock level: 45

PRODUCT 2
Product name: Mouse
Opening stock: 100
Incoming quantity: 50
Dispatched quantity: 80
Actual closing stock: 75
Minimum stock level: 60

PRODUCT 3
Product name: Keyboard
Opening stock: 40
Incoming quantity: 10
Dispatched quantity: 20
Actual closing stock: 25
Minimum stock level: 35

========== INVENTORY RECONCILIATION ==========
Laptop: Expected=40, Actual=38, Difference=-2, Replenish=7, SHORTAGE
Mouse: Expected=70, Actual=75, Difference=5, Replenish=0, EXCESS
Keyboard: Expected=30, Actual=25, Difference=-5, Replenish=10, SHORTAGE
Total expected stock: 140
Total actual stock: 138
Total replenishment required: 17
```

10. Conclusion

The program reconciles inventory using stock movement calculations, identifies discrepancies, and computes replenishment quantities based on actual closing stock.

CASE STUDY 61: Nested List Examination Seating Validation

1. Problem Statement

Validate a two-dimensional examination seating matrix to detect duplicate roll numbers, occupied reserved positions, empty seats, and invalid student allocations.

2. Objectives

• Represent seating using a nested list.

• Detect duplicate positive roll numbers.

• Identify occupied reserved seats.

• Identify empty seats and negative invalid allocations.

• Generate a validation report.

3. Assumptions

• The matrix is rectangular.

• Positive integers are student roll numbers; 0 means empty; negative values are invalid.

• Reserved positions are entered as one-based row and column pairs.

• Reserved seats must remain empty.

• Duplicate roll numbers are reported with all their positions.

4. Algorithm

• Read rows, columns, and reserved positions.

• Read every seat value into a nested list.

• Traverse each cell and record invalid, empty, and reserved-seat violations.

• Collect positions for positive roll numbers.

• Identify roll numbers appearing more than once.

• Display the matrix and validation report.

5. Flowchart

The following traditional text flowchart shows the program sequence.

```
START
  |
Read matrix dimensions and reserved seats
  |
Enter roll numbers into nested list
  |
Traverse every seat
  |
Negative? -> Invalid allocation
Zero? -> Empty seat unless reserved
Positive? -> Record roll number position
  |
Check occupied reserved seats and duplicates
  |
Display validation report
  |
 STOP
```

6. Pseudocode

```
BEGIN
  INPUT rows, columns
  INPUT reserved positions
  CREATE seating matrix
```

```
  FOR each cell
      INPUT roll_number
      STORE in matrix
  END FOR
  FOR each cell
      IF value < 0: record invalid allocation
      ELSE IF value = 0 AND position not reserved: record empty seat
      ELSE IF value > 0:
          record position for roll number
          IF position reserved: record reserved violation
      END IF
  END FOR
  FIND roll numbers with more than one position
  DISPLAY matrix and issues
END
```

7. Python Source Code

The following program is designed to run in Python 3.

```
from collections import defaultdict

def read_int(message, minimum=None):
    while True:
        try:
            value = int(input(message))
            if minimum is None or value >= minimum:
                return value
            print(f"Enter a value >= {minimum}.")
        except ValueError:
            print("Enter a valid integer.")

def main():
    print("EXAMINATION SEATING VALIDATION SYSTEM")
    rows = read_int("Enter number of rows: ", 1)
    cols = read_int("Enter number of columns: ", 1)
    reserved = set()
    count = read_int("Enter number of reserved seats: ", 0)

    for i in range(count):
        while True:
            r = read_int(f"Reserved seat {i+1} row: ", 1)
            c = read_int(f"Reserved seat {i+1} column: ", 1)
            if r <= rows and c <= cols:
                reserved.add((r, c))
                break
            print("Position outside the seating matrix.")

    seating = []
    print("Enter roll numbers (0=empty, negative=invalid).")
    for r in range(1, rows + 1):
        row = []
        for c in range(1, cols + 1):
            row.append(read_int(f"Roll number at ({r},{c}): "))
        seating.append(row)

    positions = defaultdict(list)
    empty, invalid, reserved_violations = [], [], []

    for r, row in enumerate(seating, 1):
        for c, value in enumerate(row, 1):
            pos = (r, c)
            if value < 0:
                invalid.append(pos)
            elif value == 0:
                if pos not in reserved:
                    empty.append(pos)
            else:
                positions[value].append(pos)
                if pos in reserved:
                    reserved_violations.append((value, pos))

    duplicates = {roll: pos for roll, pos in positions.items()
                  if len(pos) > 1}

    print("\n========== SEATING MATRIX ==========")
    for row in seating:
        print(" ".join(f"{value:>6}" for value in row))
    print("\nDuplicate Roll Numbers:", duplicates or "None")
    print("Occupied Reserved Seats:", reserved_violations or "None")
    print("Empty Seats:", empty or "None")
    print("Invalid Allocations:", invalid or "None")
    if duplicates or reserved_violations or empty or invalid:
        print("SEATING ARRANGEMENT REQUIRES CORRECTION")
    else:
        print("VALID SEATING ARRANGEMENT")

if __name__ == "__main__":
    main()
```

8. Test Cases

Test case	Input condition	Expected result
1	All positive roll numbers unique; reserved seats empty	Valid arrangement.
2	Same positive roll number appears twice	Duplicate detected with positions.
3	Positive roll number occupies reserved seat	Reserved-seat violation.
4	Zero in a non-reserved seat	Empty seat reported.
5	Negative roll number	Invalid allocation reported.

9. Sample Output

Illustrative execution:

```
EXAMINATION SEATING VALIDATION SYSTEM
Enter number of rows: 2
Enter number of columns: 4
Enter number of reserved seats: 1
Reserved seat 1 row: 1
Reserved seat 1 column: 3
Enter roll numbers (0=empty, negative=invalid).
Roll number at (1,1): 101
Roll number at (1,2): 102
Roll number at (1,3): 999
Roll number at (1,4): 0
Roll number at (2,1): 103
Roll number at (2,2): 101
Roll number at (2,3): -1
Roll number at (2,4): 105

========== SEATING MATRIX ==========
   101    102    999      0
   103    101     -1    105

Duplicate Roll Numbers: {101: [(1, 1), (2, 2)]}
Occupied Reserved Seats: [(999, (1, 3))]
Empty Seats: [(1, 4)]
Invalid Allocations: [(2, 3)]
SEATING ARRANGEMENT REQUIRES CORRECTION
```

10. Conclusion

The nested-list validator checks every seat, identifies duplicate roll numbers and allocation problems, and provides a report that can be used to correct the seating plan.

CASE STUDY 62: Water Distribution Fairness Analysis

1. Problem Statement

Analyse water demand and supply across several areas over multiple days, calculate shortages, compare distribution levels, and identify the most underserved area.

2. Objectives

• Store daily demand and supply for each area.

• Calculate area-wise total demand, supply, and shortage.

• Calculate shortage and fulfilment percentages.

• Identify the most underserved area.

3. Assumptions

• Quantities are measured in litres.

• Demand and supply are non-negative numbers.

• Shortage for a day is max(0, demand - supply).

• Most underserved means the highest total-shortage percentage.

• If total demand is zero, shortage percentage is defined as zero and fulfilment as 100%.

4. Algorithm

• Read area and day counts.

• For each area, read a unique name and daily demand/supply.

• Calculate daily shortages.

• Calculate totals and percentages for each area.

• Find the area with the highest shortage percentage.

• Display comparisons and result.

5. Flowchart

The following traditional text flowchart shows the program sequence.

```
START
  |
Read area count and number of days
  |
Read daily demand and supply for every area
  |
Calculate daily and total shortages
  |
Calculate shortage percentages
  |
Compare areas
  |
Identify highest shortage percentage
  |
Display distribution report
  |
 STOP
```

6. Pseudocode

```
BEGIN
  INPUT area_count, day_count
  FOR each area
      INPUT unique area name
      FOR each day
          INPUT demand, supply
          VALIDATE both >= 0
      END FOR
      shortage = SUM(MAX(0, demand - supply))
```

```
      total_demand = SUM(daily demand)
      shortage_pct = shortage / total_demand * 100
      IF total_demand = 0: shortage_pct = 0
  END FOR
  underserved = area with maximum shortage_pct
  DISPLAY report
END
```

7. Python Source Code

The following program is designed to run in Python 3.

```
def read_int(message, minimum=0):
    while True:
        try:
            value = int(input(message))
            if value >= minimum:
                return value
            print(f"Enter a value >= {minimum}.")
        except ValueError:
            print("Enter a valid integer.")

def read_number(message):
    while True:
        try:
            value = float(input(message))
            if value >= 0:
                return value
            print("Quantity cannot be negative.")
        except ValueError:
            print("Enter a valid number.")

def main():
    print("WATER DISTRIBUTION FAIRNESS ANALYSIS")
    area_count = read_int("Enter number of areas: ", 1)
    days = read_int("Enter number of days: ", 1)
    names, demands, supplies = [], [], []

    for i in range(area_count):
        print(f"\nAREA {i+1}")
        while True:
            name = input("Enter area name: ").strip()
            if name and name.casefold() not in [n.casefold() for n in names]:
                break
            print("Area name must be non-empty and unique.")
        daily_demand, daily_supply = [], []
        for day in range(days):
            print(f"Day {day+1}")
            daily_demand.append(read_number("Water demand (litres): "))
            daily_supply.append(read_number("Water supplied (litres): "))
        names.append(name)
        demands.append(daily_demand)
        supplies.append(daily_supply)

    report = []
    for i, name in enumerate(names):
        total_demand = sum(demands[i])
        total_supply = sum(supplies[i])
        shortage = sum(max(0, d-s) for d, s in zip(demands[i], supplies[i]))
        shortage_pct = shortage / total_demand * 100 if total_demand else 0
        fulfilment = (min(total_supply, total_demand) / total_demand * 100
                      if total_demand else 100)
        report.append((name, total_demand, total_supply,
                       shortage, shortage_pct, fulfilment))

    print("\n========== WATER DISTRIBUTION REPORT ==========")
    print(f"{'Area':<12}{'Demand':>10}{'Supply':>10}{'Shortage':>12}{'Shortage %':>13}")
    for name, demand, supply, shortage, pct, fulfilment in report:
        print(f"{name:<12}{demand:>10.2f}{supply:>10.2f}"
              f"{shortage:>12.2f}{pct:>12.2f}%")
        print(f"  Fulfilment: {fulfilment:.2f}%")

    underserved = max(report, key=lambda item: item[4])
    print("\nMost Underserved Area:", underserved[0])
    print(f"Shortage Percentage: {underserved[4]:.2f}%")

if __name__ == "__main__":
    main()
```

8. Test Cases

Test case	Input condition	Expected result
1	Supply equals demand for every day	No shortage.
2	Supply lower than demand	Shortage calculated.

Test case	Input condition	Expected result
3	Supply exceeds demand	Shortage is zero for that day.
4	Several areas have different shortage percentages	Highest percentage area identified.
5	Negative demand or supply	Input rejected.

9. Sample Output

Illustrative execution:

```
WATER DISTRIBUTION FAIRNESS ANALYSIS
Enter number of areas: 3
Enter number of days: 3

AREA 1
Enter area name: North
Day 1
Water demand (litres): 100
Water supplied (litres): 90
Day 2
Water demand (litres): 110
Water supplied (litres): 100
Day 3
Water demand (litres): 120
Water supplied (litres): 100

AREA 2
Enter area name: South
Day 1
Water demand (litres): 80
Water supplied (litres): 70
Day 2
Water demand (litres): 90
Water supplied (litres): 60
Day 3
Water demand (litres): 100
Water supplied (litres): 80

AREA 3
Enter area name: East
Day 1
Water demand (litres): 120
Water supplied (litres): 115
Day 2
Water demand (litres): 120
Water supplied (litres): 110
Day 3
Water demand (litres): 130
Water supplied (litres): 125

========== WATER DISTRIBUTION REPORT ==========
Area            Demand    Supply    Shortage   Shortage %
North           330.00    290.00       40.00       12.12%
  Fulfilment: 87.88%
South           270.00    210.00       60.00       22.22%
  Fulfilment: 77.78%
East            370.00    350.00       20.00        5.41%
  Fulfilment: 94.59%

Most Underserved Area: South
Shortage Percentage: 22.22%
```

10. Conclusion

The program compares water demand and supply across areas and days, quantifies shortages, and identifies the area with the greatest shortage percentage.

CASE STUDY 63: Multi-Product Price Fluctuation Analysis

1. Problem Statement

Process historical prices for multiple products, calculate percentage changes between consecutive periods, identify maximum fluctuations, and flag products exceeding a supplied threshold.

2. Objectives

• Store historical prices in nested lists.

• Calculate consecutive percentage changes.

• Find each product's maximum absolute fluctuation.

• Flag products exceeding the threshold.

• Identify the product with the highest fluctuation.

3. Assumptions

• Each product has at least two positive historical prices.

• All products use the same number of periods.

• Threshold is a non-negative percentage.

• Fluctuation means absolute percentage change, so increases and decreases are both considered.

• A product is flagged only when its maximum fluctuation is strictly greater than the threshold.

4. Algorithm

• Read product count, period count, and threshold.

• Read unique product names and positive prices.

• Calculate percentage changes between consecutive prices.

• Find maximum absolute change for each product.

• Compare with threshold and flag as required.

• Display product analysis and highest fluctuation.

5. Flowchart

The following traditional text flowchart shows the program sequence.

```
START
  |
Read product count, period count, threshold
  |
Read product names and historical prices
  |
Calculate consecutive percentage changes
  |
Find maximum absolute change per product
  |
Greater than threshold?
  | YES -> Flag product
  | NO  -> Within threshold
  |
Identify maximum fluctuation product
  |
Display report
  |
 STOP
```

6. Pseudocode

```
BEGIN
  INPUT product_count, period_count (at least 2), threshold
  FOR each product
      INPUT unique product name
```

```
      INPUT positive price for each period
      FOR each consecutive price pair
          change = (current - previous) / previous * 100
      END FOR
      maximum = MAX(ABS(change))
      IF maximum > threshold: flag product
  END FOR
  DISPLAY changes, flags, and highest fluctuation
END
```

7. Python Source Code

The following program is designed to run in Python 3.

```
def read_int(message, minimum=1):
    while True:
        try:
            value = int(input(message))
            if value >= minimum:
                return value
            print(f"Enter an integer >= {minimum}.")
        except ValueError:
            print("Enter a valid integer.")

def read_positive_float(message):
    while True:
        try:
            value = float(input(message))
            if value > 0:
                return value
            print("Price must be greater than zero.")
        except ValueError:
            print("Enter a valid number.")

def main():
    print("MULTI-PRODUCT PRICE FLUCTUATION ANALYSIS")
    product_count = read_int("Enter number of products: ")
    periods = read_int("Enter number of historical periods (minimum 2): ", 2)
    while True:
        try:
            threshold = float(input("Enter fluctuation threshold (%): "))
            if threshold >= 0:
                break
            print("Threshold cannot be negative.")
        except ValueError:
            print("Enter a valid percentage.")

    names, histories = [], []
    for i in range(product_count):
        print(f"\nPRODUCT {i+1}")
        while True:
            name = input("Enter product name: ").strip()
            if name and name.casefold() not in [n.casefold() for n in names]:
                break
            print("Product name must be non-empty and unique.")
        prices = [read_positive_float(f"Enter price for period {p+1}: ")
                  for p in range(periods)]
        names.append(name)
        histories.append(prices)

    report = []
    for name, prices in zip(names, histories):
        changes = [
            (prices[i] - prices[i-1]) / prices[i-1] * 100
            for i in range(1, len(prices))
        ]
        maximum = max(abs(change) for change in changes)
        report.append((name, prices, changes, maximum, maximum > threshold))

    print("\n========== PRICE FLUCTUATION REPORT ==========")
    print(f"Threshold: {threshold:.2f}%")
    for name, prices, changes, maximum, flagged in report:
        print(f"\nProduct: {name}")
        print("Historical Prices:", prices)
        for i, change in enumerate(changes, 1):
            print(f"Period {i} to {i+1}: {change:+.2f}%")
        print(f"Maximum Fluctuation: {maximum:.2f}%")
        print("Status:", "FLAGGED" if flagged else "WITHIN THRESHOLD")

    highest = max(report, key=lambda item: item[3])
    flagged_names = [item[0] for item in report if item[4]]
    print("\nHighest Fluctuation Product:", highest[0])
    print(f"Maximum Fluctuation: {highest[3]:.2f}%")
    print("Flagged Products:", flagged_names)

if __name__ == "__main__":
    main()
```

8. Test Cases

Test case	Input condition	Expected result
1	Prices unchanged between periods	0% change; not flagged for positive threshold.
2	Price increases by 10%	+10% change.
3	Price decreases by 12.5%	-12.5% change; absolute fluctuation 12.5%.
4	Maximum fluctuation exceeds threshold	Product flagged.
5	Zero or negative price	Input rejected.

9. Sample Output

Illustrative execution:

```
MULTI-PRODUCT PRICE FLUCTUATION ANALYSIS
Enter number of products: 3
Enter number of historical periods (minimum 2): 4
Enter fluctuation threshold (%): 10

PRODUCT 1
Enter product name: Laptop
Enter price for period 1: 50000
Enter price for period 2: 52000
Enter price for period 3: 49000
Enter price for period 4: 55000

PRODUCT 2
Enter product name: Phone
Enter price for period 1: 20000
Enter price for period 2: 19800
Enter price for period 3: 20200
Enter price for period 4: 20400

PRODUCT 3
Enter product name: Tablet
Enter price for period 1: 15000
Enter price for period 2: 16500
Enter price for period 3: 16000
Enter price for period 4: 14000

========== PRICE FLUCTUATION REPORT ==========
Threshold: 10.00%

Product: Laptop
Historical Prices: [50000.0, 52000.0, 49000.0, 55000.0]
Period 1 to 2: +4.00%
Period 2 to 3: -5.77%
Period 3 to 4: +12.24%
Maximum Fluctuation: 12.24%
Status: FLAGGED

Product: Phone
Historical Prices: [20000.0, 19800.0, 20200.0, 20400.0]
Period 1 to 2: -1.00%
Period 2 to 3: +2.02%
Period 3 to 4: +0.99%
Maximum Fluctuation: 2.02%
Status: WITHIN THRESHOLD

Product: Tablet
Historical Prices: [15000.0, 16500.0, 16000.0, 14000.0]
Period 1 to 2: +10.00%
Period 2 to 3: -3.03%
Period 3 to 4: -12.50%
Maximum Fluctuation: 12.50%
Status: FLAGGED

Highest Fluctuation Product: Tablet
Maximum Fluctuation: 12.50%
Flagged Products: ['Laptop', 'Tablet']
```

10. Conclusion

The program analyses historical prices, calculates percentage changes, and identifies products whose price movements exceed a specified threshold.

CASE STUDY 98: University Academic Progression Audit

1. Problem Statement

Combine semester marks, subject pass conditions, attendance, and prerequisite completion records to determine student progression eligibility and identify outstanding requirements.

2. Objectives

• Audit semester-wise marks.

• Check attendance against the required percentage.

• Verify prerequisite completion.

• Identify failed subjects and outstanding requirements.

• Determine progression eligibility.

3. Assumptions

• Marks are out of 100; pass mark is 35 in each semester for each subject.

• Minimum attendance is 75%.

• Attendance is entered as a percentage from 0 to 100.

• Prerequisite status is Y or N for each subject.

• A student is eligible only if all recorded requirements are satisfied.

4. Algorithm

• Read student identity and academic dimensions.

• For each subject, read semester marks, attendance, and prerequisite status.

• Validate inputs.

• Identify every mark below 35.

• Identify attendance below 75% and incomplete prerequisites.

• Determine eligibility and display audit.

5. Flowchart

The following traditional text flowchart shows the program sequence.

```
START
  |
Read student and academic details
  |
Read marks, attendance, prerequisites
  |
Check every semester mark
  |
Check attendance >= 75%
  |
Check prerequisites completed
  |
Any outstanding requirement?
  | YES -> NOT ELIGIBLE
  | NO  -> ELIGIBLE
  |
Display academic audit
  |
 STOP
```

6. Pseudocode

```
BEGIN
  INPUT student details, semester count, subject count
  FOR each subject
      INPUT subject name
```

```
      FOR each semester
          INPUT mark and validate 0..100
          IF mark < 35: record failed subject/semester
      END FOR
      INPUT attendance and validate 0..100
      IF attendance < 75: record shortage
      INPUT prerequisite status Y/N
      IF status = N: record pending prerequisite
  END FOR
  eligible = no failures AND no attendance shortages
             AND no pending prerequisites
  DISPLAY audit and eligibility
END
```

7. Python Source Code

The following program is designed to run in Python 3.

```
PASS_MARK = 35
MIN_ATTENDANCE = 75.0

def read_int(message, minimum=1):
    while True:
        try:
            value = int(input(message))
            if value >= minimum:
                return value
            print(f"Enter an integer >= {minimum}.")
        except ValueError:
            print("Enter a valid integer.")

def read_number(message, low, high):
    while True:
        try:
            value = float(input(message))
            if low <= value <= high:
                return value
            print(f"Enter a value from {low} to {high}.")
        except ValueError:
            print("Enter a valid number.")

def main():
    print("UNIVERSITY ACADEMIC PROGRESSION AUDIT")
    name = input("Enter student name: ").strip() or "Student"
    roll = input("Enter roll number: ").strip() or "N/A"
    semesters = read_int("Enter number of semesters: ")
    subject_count = read_int("Enter number of subjects: ")
    subjects, used_names = [], set()

    for i in range(subject_count):
        print(f"\nSUBJECT {i+1}")
        while True:
            subject = input("Enter subject name: ").strip()
            if subject and subject.casefold() not in used_names:
                used_names.add(subject.casefold())
                break
            print("Subject name must be non-empty and unique.")
        marks = [read_number(f"Semester {s} marks: ", 0, 100)
                 for s in range(1, semesters+1)]
        attendance = read_number("Attendance percentage: ", 0, 100)
        while True:
            prerequisite = input("Prerequisites completed? (Y/N): ").strip().upper()
            if prerequisite in ("Y", "N"):
                break
            print("Enter Y or N.")
        subjects.append((subject, marks, attendance, prerequisite))

    failed, shortages, pending = [], [], []
    for subject, marks, attendance, prerequisite in subjects:
        for semester, mark in enumerate(marks, 1):
            if mark < PASS_MARK:
                failed.append((subject, semester, mark))
        if attendance < MIN_ATTENDANCE:
            shortages.append((subject, attendance))
        if prerequisite == "N":
            pending.append(subject)

    eligible = not (failed or shortages or pending)
    print("\n========== ACADEMIC PROGRESSION AUDIT ==========")
    print(f"Student: {name} | Roll Number: {roll}")
    for subject, marks, attendance, prerequisite in subjects:
        print(f"{subject}: Marks={marks}, Attendance={attendance:.2f}%, "
              f"Prerequisites={prerequisite}")
    print("\nFailed Subjects:", failed or "None")
    print("Attendance Shortages:", shortages or "None")
    print("Pending Prerequisites:", pending or "None")
    print("PROGRESSION STATUS:", "ELIGIBLE" if eligible else "NOT ELIGIBLE")

if __name__ == "__main__":
```

```
main()
```

8. Test Cases

Test case	Input condition	Expected result
1	All marks >=35, attendance >=75%, all prerequisites Y	Eligible.
2	Any semester mark below 35	Not eligible; failed subject listed.
3	Attendance below 75%	Not eligible; attendance shortage listed.
4	Any prerequisite N	Not eligible; prerequisite listed.
5	Multiple conditions fail	Every outstanding requirement listed.

9. Sample Output

Illustrative execution:

```
UNIVERSITY ACADEMIC PROGRESSION AUDIT
Enter student name: Arjun
Enter roll number: BTECH101
Enter number of semesters: 2
Enter number of subjects: 3

SUBJECT 1
Enter subject name: Python
Semester 1 marks: 78
Semester 2 marks: 82
Attendance percentage: 90
Prerequisites completed? (Y/N): Y

SUBJECT 2
Enter subject name: Mathematics
Semester 1 marks: 30
Semester 2 marks: 65
Attendance percentage: 80
Prerequisites completed? (Y/N): Y

SUBJECT 3
Enter subject name: Physics
Semester 1 marks: 70
Semester 2 marks: 75
Attendance percentage: 68
Prerequisites completed? (Y/N): N

========== ACADEMIC PROGRESSION AUDIT ==========
Student: Arjun | Roll Number: BTECH101
Python: Marks=[78.0, 82.0], Attendance=90.00%, Prerequisites=Y
Mathematics: Marks=[30.0, 65.0], Attendance=80.00%, Prerequisites=Y
Physics: Marks=[70.0, 75.0], Attendance=68.00%, Prerequisites=N

Failed Subjects: [('Mathematics', 1, 30.0)]
Attendance Shortages: [('Physics', 68.0)]
Pending Prerequisites: ['Physics']
PROGRESSION STATUS: NOT ELIGIBLE
```

10. Conclusion

The audit combines academic marks, attendance, and prerequisite checks to produce an eligibility decision and a transparent list of outstanding requirements.

CASE STUDY 99: Integrated E-Commerce Order Processing and Reconciliation

1. Problem Statement

Process product orders, discounts, dispatches, returns, and stock updates; calculate revenue, identify discrepancies, and generate a final inventory summary.

2. Objectives

• Process product orders and discounts.

• Validate dispatch and return quantities.

• Calculate net sales revenue.

• Calculate expected closing stock and compare with physical stock.

• Report shortages, excesses, and totals.

3. Assumptions

• Product names are unique.

• Opening stock and quantities are non-negative integers; unit price is positive.

• Dispatch cannot exceed ordered quantity or opening stock.

• Returns cannot exceed dispatched quantity.

• Discount is from 0% to 100%.

• Returned units are assumed fit for restocking.

• Revenue = (dispatched - returned) × discounted unit price.

4. Algorithm

• Read product count and product details.

• Validate opening stock, price, order, and discount.

• Read dispatch and return quantities and validate limits.

• Calculate net sold quantity and discounted revenue.

• Calculate expected closing stock.

• Read physical closing stock and compute discrepancy.

• Summarize revenue, stock, shortages, and excesses.

5. Flowchart

The following traditional text flowchart shows the program sequence.

```
START
  |
Read product and order details
  |
Validate order and discount
  |
Read dispatch and return quantities
  |
Validate dispatch and returns
  |
Calculate net sold quantity and revenue
  |
Calculate expected stock
  |
Compare with physical closing stock
  |
More products?
  | YES -> Repeat
```

```
  | NO
  |
Display revenue and inventory summary
  |
 STOP
```

6. Pseudocode

```
BEGIN
  INPUT product_count
  FOR each product
      INPUT unique name, opening stock, unit price
      INPUT order quantity, discount percentage
      VALIDATE discount 0..100
      INPUT dispatched; validate <= order and opening stock
      INPUT returned; validate <= dispatched
      net_sold = dispatched - returned
      revenue = net_sold * unit_price * (1 - discount/100)
      expected = opening - dispatched + returned
      INPUT physical closing stock
      discrepancy = physical - expected
      CLASSIFY shortage/excess/matched
      STORE product data
  END FOR
  CALCULATE total revenue and stock totals
  DISPLAY report
END
```

7. Python Source Code

The following program is designed to run in Python 3.

```
def read_int(message, minimum=0):
    while True:
        try:
            value = int(input(message))
            if value >= minimum:
                return value
            print(f"Enter an integer >= {minimum}.")
        except ValueError:
            print("Enter a valid integer.")

def read_float(message, minimum=0, maximum=None, strict=False):
    while True:
        try:
            value = float(input(message))
            valid = value > minimum if strict else value >= minimum
            if valid and (maximum is None or value <= maximum):
                return value
            print("Value is outside the permitted range.")
        except ValueError:
            print("Enter a valid number.")

def main():
    print("INTEGRATED E-COMMERCE ORDER PROCESSING")
    count = read_int("Enter number of products: ", 1)
    records, names = [], set()

    for i in range(count):
        print(f"\nPRODUCT {i+1}")
        while True:
            name = input("Enter product name: ").strip()
            if name and name.casefold() not in names:
                names.add(name.casefold())
                break
            print("Product name must be non-empty and unique.")
        opening = read_int("Opening stock: ")
        price = read_float("Unit price: ", 0, strict=True)
        ordered = read_int("Ordered quantity: ", 1)
        discount = read_float("Discount percentage: ", 0, 100)

        while True:
            dispatched = read_int("Dispatched quantity: ")
            if dispatched <= ordered and dispatched <= opening:
                break
            print("Dispatch cannot exceed order or opening stock.")
        while True:
            returned = read_int("Returned quantity: ")
            if returned <= dispatched:
                break
            print("Returns cannot exceed dispatched quantity.")

        physical = read_int("Physical closing stock: ")
        net_sold = dispatched - returned
        revenue = net_sold * price * (1 - discount / 100)
        expected = opening - dispatched + returned
        discrepancy = physical - expected
```

```
        status = ("SHORTAGE" if discrepancy < 0 else
                  "EXCESS" if discrepancy > 0 else "MATCHED")
        records.append((name, ordered, dispatched, returned, revenue,
                        expected, physical, discrepancy, status))

    print("\n========== E-COMMERCE ORDER REPORT ==========")
    for r in records:
        name, ordered, dispatched, returned, revenue, expected, physical, diff, status = r
        print(f"{name}: Ordered={ordered}, Dispatched={dispatched}, "
              f"Returned={returned}, Revenue=Rs.{revenue:.2f}")
        print(f"Expected stock={expected}, Physical stock={physical}, "
              f"Difference={diff}, Status={status}")

    total_revenue = sum(r[4] for r in records)
    total_expected = sum(r[5] for r in records)
    total_physical = sum(r[6] for r in records)
    print("\n========== FINAL INVENTORY SUMMARY ==========")
    print(f"Total Revenue: Rs. {total_revenue:.2f}")
    print(f"Expected Closing Stock: {total_expected}")
    print(f"Physical Closing Stock: {total_physical}")
    print(f"Overall Stock Discrepancy: {total_physical-total_expected}")
    print("Matched Products:", sum(r[8] == "MATCHED" for r in records))
    print("Products with Shortage:", sum(r[8] == "SHORTAGE" for r in records))
    print("Products with Excess:", sum(r[8] == "EXCESS" for r in records))

if __name__ == "__main__":
    main()
```

8. Test Cases

Test case	Input condition	Expected result
1	Valid order, no discount	Revenue at original price.
2	Valid order with discount	Revenue reduced by discount.
3	Returned quantity less than dispatched	Net sales and revenue reduced; returns added back to stock.
4	Dispatch greater than order or opening stock	Input rejected.
5	Return greater than dispatched quantity	Input rejected.
6	Physical stock lower/higher/equal to expected	Shortage/excess/matched status.

9. Sample Output

Illustrative execution:

```
INTEGRATED E-COMMERCE ORDER PROCESSING
Enter number of products: 2

PRODUCT 1
Enter product name: Laptop
Opening stock: 50
Unit price: 50000
Ordered quantity: 10
Discount percentage: 10
Dispatched quantity: 8
Returned quantity: 1
Physical closing stock: 43

PRODUCT 2
Enter product name: Mouse
Opening stock: 100
Unit price: 500
Ordered quantity: 20
Discount percentage: 5
Dispatched quantity: 15
Returned quantity: 2
Physical closing stock: 88

========== E-COMMERCE ORDER REPORT ==========
Laptop: Ordered=10, Dispatched=8, Returned=1, Revenue=Rs.315000.00
Expected stock=43, Physical stock=43, Difference=0, Status=MATCHED
Mouse: Ordered=20, Dispatched=15, Returned=2, Revenue=Rs.6175.00
Expected stock=87, Physical stock=88, Difference=1, Status=EXCESS

========== FINAL INVENTORY SUMMARY ==========
Total Revenue: Rs. 321175.00
Expected Closing Stock: 130
Physical Closing Stock: 131
Overall Stock Discrepancy: 1
Matched Products: 1
Products with Shortage: 0
Products with Excess: 1
```

10. Conclusion

The program integrates order processing, discounts, dispatches, returns, revenue calculations, and inventory reconciliation to provide a consolidated business report.

CASE STUDY 100: Comprehensive Smart Campus Management Simulation

1. Problem Statement

Develop a menu-driven campus application integrating student records, attendance, marks, event registration, resource allocation, and summary reporting using conditional statements, loops, strings, lists, nested lists, tuples, and tuple unpacking.

2. Objectives

• Manage student records.

• Record attendance and calculate attendance percentage.

• Store subject-wise marks.

• Register students for campus events.

• Allocate limited campus resources.

• Generate an integrated summary.

• Demonstrate required Python data structures and control statements.

3. Assumptions

• Student IDs are unique.

• Student record is stored as a tuple (name, department, year).

• Attendance is stored as [attended, total].

• Marks are stored as nested lists [subject, marks].

• Each student can register once per event.

• Events and resources have fixed capacities.

• All data is held in memory and resets when the program exits.

4. Algorithm

• Initialize student, attendance, marks, event, and resource collections.

• Display menu repeatedly.

• Execute the selected operation: student registration, attendance, marks, event registration, resource allocation, or report.

• Validate records and capacities.

• Repeat until Exit is selected.

• Display confirmation or error messages for each operation.

5. Flowchart

The following traditional text flowchart shows the program sequence.

```
START
  |
Initialize campus data
  |
Display main menu
  |
Read menu choice
  |
Choose operation:
  Student / Attendance / Marks /
  Event / Resource / Summary
  |
```

```
Validate and update selected records
  |
Display operation result
  |
Exit selected?
  | NO -> Return to menu
  | YES
  |
 STOP
```

6. Pseudocode

```
BEGIN
  INITIALIZE all campus collections
  REPEAT
      DISPLAY menu
      INPUT choice
      IF choice = student: add unique student tuple
      ELSE IF choice = attendance: validate and update sessions
      ELSE IF choice = marks: add subject marks
      ELSE IF choice = event: check capacity and duplicate
      ELSE IF choice = resource: check available capacity
      ELSE IF choice = summary: display all records
      ELSE IF choice = exit: terminate
      ELSE: display invalid choice
  UNTIL choice = exit
END
```

7. Python Source Code

The following program is designed to run in Python 3.

```
students = {}
attendance = {}
marks = {}
events = {
    "Technical Fest": [50, []],
    "Cultural Fest": [40, []],
    "Sports Meet": [30, []]
}
resources = {
    "Projector": [10, 0],
    "Computer": [50, 0],
    "Laboratory": [5, 0]
}

def read_int(message, minimum=0):
    while True:
        try:
            value = int(input(message))
            if value >= minimum:
                return value
            print(f"Enter a value >= {minimum}.")
        except ValueError:
            print("Enter a valid integer.")

def read_text(message):
    while True:
        value = input(message).strip()
        if value:
            return value
        print("This field cannot be empty.")

def add_student():
    sid = read_text("Student ID: ")
    if sid in students:
        print("Student ID already exists.")
        return
    name = read_text("Student name: ")
    department = read_text("Department: ")
    year = read_int("Academic year (1-4): ", 1)
    if year > 4:
        print("Academic year must be from 1 to 4.")
        return
    students[sid] = (name, department, year)
    attendance[sid] = [0, 0]
    marks[sid] = []
    print("Student registered successfully.")

def update_attendance():
    sid = read_text("Student ID: ")
    if sid not in students:
        print("Student not found.")
        return
    total = read_int("Additional total sessions: ", 1)
    attended = read_int("Attended sessions: ")
    if attended > total:
```

```
        print("Attended sessions cannot exceed total sessions.")
        return
    attendance[sid][0] += attended
    attendance[sid][1] += total
    a, t = attendance[sid]
    print(f"Attendance updated: {a/t*100:.2f}%")

def add_marks():
    sid = read_text("Student ID: ")
    if sid not in students:
        print("Student not found.")
        return
    subject = read_text("Subject name: ")
    if any(item[0].casefold() == subject.casefold() for item in marks[sid]):
        print("Marks for this subject already exist.")
        return
    score = read_int("Marks (0-100): ")
    if score > 100:
        print("Marks must not exceed 100.")
        return
    marks[sid].append([subject, score])
    print("Marks added successfully.")

def register_event():
    sid = read_text("Student ID: ")
    if sid not in students:
        print("Student not found.")
        return
    print("Available events:", ", ".join(events))
    event = read_text("Event name: ")
    if event not in events:
        print("Event does not exist.")
        return
    capacity, registered = events[event]
    if sid in registered:
        print("Student already registered.")
    elif len(registered) >= capacity:
        print("Event is full.")
    else:
        registered.append(sid)
        print("Event registration successful.")

def allocate_resource():
    print("Available resources:", ", ".join(resources))
    name = read_text("Resource name: ")
    if name not in resources:
        print("Resource does not exist.")
        return
    quantity = read_int("Required quantity: ", 1)
    capacity, allocated = resources[name]
    if quantity > capacity - allocated:
        print("Insufficient resource capacity.")
        return
    resources[name][1] += quantity
    print("Resource allocated successfully.")

def display_summary():
    print("\n========== SMART CAMPUS SUMMARY ==========")
    for sid, record in students.items():
        # Tuple unpacking
        name, department, year = record
        attended, total = attendance[sid]
        percentage = attended / total * 100 if total else 0
        print(f"\n{sid}: {name}, {department}, Year {year}")
        print(f"Attendance: {attended}/{total} ({percentage:.2f}%)")
        print("Marks:", marks[sid] if marks[sid] else "No marks")
    print("\nEVENTS")
    for event, (capacity, registered) in events.items():
        print(f"{event}: {len(registered)}/{capacity}")
    print("\nRESOURCES")
    for name, (capacity, allocated) in resources.items():
        print(f"{name}: allocated={allocated}, available={capacity-allocated}")

def main():
    while True:
        print("\n===== SMART CAMPUS MANAGEMENT =====")
        print("1. Student Management")
        print("2. Attendance Management")
        print("3. Marks Management")
        print("4. Event Registration")
        print("5. Resource Allocation")
        print("6. Campus Summary Report")
        print("0. Exit")
        choice = input("Enter your choice: ").strip()

        if choice == "1":
            add_student()
        elif choice == "2":
            update_attendance()
        elif choice == "3":
```

```
            add_marks()
        elif choice == "4":
            register_event()
        elif choice == "5":
            allocate_resource()
        elif choice == "6":
            display_summary()
        elif choice == "0":
            print("Thank you for using Smart Campus Management System.")
            break
        else:
            print("Invalid choice. Select an option from the menu.")

if __name__ == "__main__":
    main()
```

8. Test Cases

Test case	Input condition	Expected result
1	Register a new student with unique ID	Student record created.
2	Register an existing student ID again	Duplicate rejected.
3	Attendance attended count exceeds total	Update rejected.
4	Marks outside 0..100	Input rejected.
5	Register same student twice for one event	Duplicate registration rejected.
6	Allocate more than remaining resource capacity	Allocation rejected.
7	Select summary report	All stored campus information displayed.
8	Select 0	Program exits.

9. Sample Output

Illustrative execution:

```
===== SMART CAMPUS MANAGEMENT =====
1. Student Management
2. Attendance Management
3. Marks Management
4. Event Registration
5. Resource Allocation
6. Campus Summary Report
0. Exit
Enter your choice: 1
Student ID: ST101
Student name: Arjun
Department: CSE
Academic year (1-4): 1
Student registered successfully.

Enter your choice: 2
Student ID: ST101
Additional total sessions: 100
Attended sessions: 90
Attendance updated: 90.00%

Enter your choice: 3
Student ID: ST101
Subject name: Python
Marks (0-100): 88
Marks added successfully.

Enter your choice: 4
Student ID: ST101
Available events: Technical Fest, Cultural Fest, Sports Meet
Event name: Technical Fest
Event registration successful.

Enter your choice: 5
Available resources: Projector, Computer, Laboratory
Resource name: Projector
Required quantity: 2
Resource allocated successfully.

Enter your choice: 6

========== SMART CAMPUS SUMMARY ==========

ST101: Arjun, CSE, Year 1
Attendance: 90/100 (90.00%)
```

```
Marks: [['Python', 88]]

EVENTS
Technical Fest: 1/50
Cultural Fest: 0/40
Sports Meet: 0/30

RESOURCES
Projector: allocated=2, available=8
Computer: allocated=0, available=50
Laboratory: allocated=0, available=5

Enter your choice: 0
Thank you for using Smart Campus Management System.
```

10. Conclusion

The smart campus application integrates multiple campus activities in one menu-driven program and demonstrates core Python data structures, validation, loops, and conditional logic.

PYTHON PROGRAMMING

CASE STUDIES – COMPLETE COLLECTION

Ten case studies with problem statements, objectives, assumptions, algorithms, flowcharts, pseudocode, Python source code, test cases, sample outputs, and conclusions.

Prepared as a structured academic assignment
Python 3 | College Practical / Case Study Record

TABLE OF CONTENTS

Case study number	Title
1	Examination Result and Grace-Mark Validation
12	Attendance Recovery Computation
13	ATM Transaction Sequence Simulation
4	Warehouse Inventory Reconciliation
61	Nested List Examination Seating Validation
62	Water Distribution Fairness Analysis
63	Multi-Product Price Fluctuation Analysis
98	University Academic Progression Audit
99	Integrated E-Commerce Order Processing and Reconciliation
100	Comprehensive Smart Campus Management Simulation

Each case study follows the same ten-section structure. The flowcharts are presented in a traditional text-based sequence for easy reading and printing.